UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 5

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SAZID ABDULLAH

STUDENT ID : 23101146

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 8/29/2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

1. Problem:

Given the following processes with their **Arrival Time (AT)** and **Burst Time (BT)**:

Process	Arrival Time (AT)	Burst Time (BT)
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

Implement the **Preemptive Shortest Job First (SJF)** CPU Scheduling Algorithm using a **Time Quantum (TQ) of 2 units**.

Calculate the following for each process:

- **Completion Time (CT)**
- **Turnaround Time (TAT)**
- **Waiting Time (WT)**

Also, calculate:

- **Average Turnaround Time (Average TAT)**
- **Average Waiting Time (Average WT)**

Finally, display the results for all processes.

2. Source Code:

```
process = ["P1", "P2", "P3", "P4", "P5"]

AT = [4, 2, 1, 0, 3]
BT = [2, 2, 3, 6, 1]

n = len(process)
quantum = 2

remaining = BT.copy()

CT = [0] * n
TAT = [0] * n
WT = [0] * n

time = 0
completed = 0

while completed < n:

    x = -1


    for i in range(n):

        if AT[i] <= time and remaining[i] > 0:

            if x == -1 or remaining[i] < remaining[x]:
                x = i


    if x == -1:
        time += 1

    else:


        if remaining[x] > quantum:
```

```
            time += quantum
            remaining[x] -= quantum

        else:
            time += remaining[x]
            remaining[x] = 0

            CT[x] = time
            completed += 1




for i in range(n):

    TAT[i] = CT[i] - AT[i]
    WT[i] = TAT[i] - BT[i]



print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):

    print(process[i], "\t",
          AT[i], "\t",
          BT[i], "\t",
          CT[i], "\t",
          TAT[i], "\t",
          WT[i])


avg_tat = sum(TAT) / n
avg_wt = sum(WT) / n

print("\nAverage TAT =", avg_tat)
print("Average WT =", avg_wt)
```

3. Output:

```
Process AT      BT      CT      TAT     WT
p1      4       2       7       3       1
p2      2       2       4       2       0
p3      1       3       10      9       6
p4      0       6       14      14      8
p5      3       1       5       2       1

Average TAT = 6.0
Average WT = 3.2


PS G:\All Notes\Machine Learning>
```

4. Discussion:

In this experiment, Preemptive Shortest Job First (SJF) and Non-Preemptive Shortest Job First (SJF) CPU scheduling techniques were applied to the same five processes. Their performances were compared by analyzing Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT, and Average WT.

The Preemptive SJF approach achieved an Average TAT of 6.0 and an Average WT of 3.2. In this scheduling method, the CPU can switch from the currently running process to another process if the newly arrived process has a shorter remaining burst time. As a result, processes requiring less CPU time can be completed sooner.

In contrast, the Non-Preemptive SJF method resulted in an Average TAT of 7.4 and an Average WT of 4.6. Here, a process keeps the CPU until its execution is finished, even if a shorter process arrives during that time. This may cause shorter processes to remain in the ready queue for a longer period.

From the obtained results, it is clear that Preemptive SJF achieved lower average turnaround and waiting times than Non-Preemptive SJF. Thus, for the selected process set, Preemptive SJF demonstrates better scheduling efficiency.

5. Conclusion:

The experiment was carried out successfully to implement and compare Preemptive and Non-Preemptive SJF scheduling algorithms. Based on the calculated results, Preemptive SJF provided an Average TAT of 6.0 and an Average WT of 3.2, whereas Non-Preemptive SJF produced an Average TAT of 7.4 and an Average WT of 4.6.

These results indicate that Preemptive SJF is more efficient for this particular set of processes because it reduces both turnaround time and waiting time. However, Non-Preemptive SJF has the advantage of being easier and more straightforward since a process is allowed to complete without interruption once it begins execution.

6. GitHub: https://github.com/AbirOPS2/CSE402-Operating-System-Lab/tree/main/Lab-05-SJF-Preemptive